შემაჯამებელი კითხვარი

პროგრამა Front-end პროგრამირება

მოდული პრაქტიკული პროექტი (06.1.1.9)

სტუდენტი იობი შარაშენიძე

პროექტი WMS — საწყობის მართვის სისტემა (https://finally-project-3lf1.vercel.app)

ბლოკი I: პროექტის დაგეგმვა, რისკები და ტექნოლოგიური სტეკი

1. სამუშაო გეგმა და Milestones

სამუშაოს ეტაპებად დაყოფა თავიდანვე აჩენს პროექტის მოცულობას, ამოცანების თანმიმდევრობასა და ვადებს. ჩემი პროექტი ოთხ ეტაპად დავყავი: დაგეგმვა და არქიტექტურა → UI და Layout → CRUD, State და ავტორიზაცია → ოპტიმიზაცია, ტესტირება და დოკუმენტაცია; ამოცანებს GitHub Projects-ის კანბან-დაფაზე, Story Points-ით ვმართავდი. დაგეგმვის გარეშე კოდის წერისას სტრუქტურა ქაოტური ხდება, ერთი და იგივე ნაწილი რამდენჯერმე გადასაკეთებელია და ვადები ცდება, რადგან დამოკიდებულებები (მაგალითად, ჯერ როლები, მერე დაცული გვერდები) დროულად არ ჩანს.

2. რისკების ანალიზი

პირველი ტექნიკური რისკი იყო მონაცემების დაკარგვა გვერდის განახლებისას: ამიტომ გლობალური Store localStorage-თან სინქრონდება, JWT სესია კი ცალკე ინახება. მეორე რისკი იყო ორმაგი დაჭერა და ერთდროული ჩამოწერა (Race Condition): გაგზავნისას ღილაკი იბლოკება, Zod ამოწმებს, რომ რაოდენობა ნაშთს არ აღემატებოდეს, ხოლო ჩაწერამდე სერვერიდან უახლესი ნაშთი მოწმდება. დროითი რისკი იყო დიდი მოცულობა ერთი დეველოპერისთვის: ჯერ MVP (ავტორიზაცია, მარაგები, CRUD) დავასრულე და დამატებითი მოდულები (რუკა, სტატისტიკა, შეტყობინებები) მხოლოდ ამის შემდეგ ავაწყე.

3. ტექნოლოგიური სტეკის შერჩევა

სტეკს ოთხი კრიტერიუმით ვარჩევდი: კურსის მოთხოვნები, ჩემი ცოდნის დონე, საზოგადოების მხარდაჭერა და საბოლოო ხარისხი. ავირჩიე React 19 + Vite (კომპონენტები და სწრაფი build), TypeScript (შეცდომები გაშვებამდე ჩანს), Tailwind CSS და CSS ცვლადები (დიზაინ-ტოკენები ერთ ადგილას), Context API + useReducer (გლობალური Store დამატებითი ბიბლიოთეკის გარეშე) და React Hook Form + Zod (ფორმები და ვალიდაცია). ბექენდად json-server და json-server-auth (JWT) ავიღე, რათა ძირითადი დრო ფრონტენდს დავთმობოდი, ჰოსტინგად კი Vercel.

ბლოკი II: არქიტექტურა, სემანტიკური HTML5 და Git

4. სემანტიკური მარკირება (HTML5)

გვერდის სტრუქტურა სემანტიკური თეგებით ავაწყე: <header> — ზედა ზოლი, <aside> და <nav aria-label> — მენიუ, <main> — გვერდის შიგთავსი, <section> — ბლოკები (სტატისტიკა, მოთხოვნები), <article> — ბარათები, <footer> — ქვედა ზოლი; ცხრილებში <th scope>, ნივთის მონაცემებისთვის კი <dl> გამოვიყენე. ეს თეგები ეკრანის წამკითხველს (Screen Reader) გვერდის სტრუქტურის გაგებასა და ნავიგაციაში ეხმარება, საძიებო სისტემებს კი შიგთავსის სწორად ინდექსირებაში. შედეგად Lighthouse-ის Accessibility-ისა და SEO-ს ქულა 100-ია.

5. მოდულური არქიტექტურა (SOLID / DRY)

საქაღალდეები პასუხისმგებლობის მიხედვით დავყავი: components/ (UI ნაწილები), pages/ (გვერდები), hooks/ (ლოგიკა), store/ და context/ (გლობალური მდგომარეობა), services/ (HTTP მოთხოვნები), config/ (როლები, საწყობები, სტატუსები), types/ და schemas/. ამის წყალობით ერთი ფუნქცია ერთხელ იწერება და ბევრგან გამოიყენება (DRY): request() ყველა API მოთხოვნას ემსახურება, StatCard 12 ადგილას გამოიყენება, FormField კი ყველა ფორმაში. კომპონენტებში fetch არ წერია, ამიტომ API-ს შეცვლისას მხოლოდ services/ იცვლება — ეს SOLID-ის Dependency Inversion პრინციპია.

6. Git და Branching

ცალკე feature/ ბრენჩზე მუშაობისას main ყოველთვის სტაბილური რჩება: ცვლილება Pull Request-ით მოწმდება და მხოლოდ ამის შემდეგ ერთიანდება, საჭიროების შემთხვევაში კი ადვილად უქმდება. ჩემს პროექტში ფინალური დოკუმენტაცია docs/final-readme ბრენჩზე მომზადდა და main-ს Pull Request #1-ით შეუერთდა. Conventional Commits არის commit-ის შეტყობინების სტანდარტი, სადაც პრეფიქსი ცვლილების ტიპს აჩვენებს: feat: — ახალი ფუნქცია, fix: — შეცდომის გასწორება, refactor: — კოდის გადაწყობა, perf: — ოპტიმიზაცია, docs: — დოკუმენტაცია; ჩემს რეპოზიტორიაში 96 commit-დან 85 ამ სტანდარტს მიჰყვება.

ბლოკი III: გლობალური სტილები, დიზაინ-სისტემა და Layout

7. CSS ცვლადები და დიზაინ-ტოკენები

გლობალურ ცვლადებში (:root, Tailwind-ის @theme) ფერი, შრიფტი და დაშორება ერთხელ იწერება, ამიტომ ერთ ადგილას შეცვლა მთელ საიტზე აისახება და დიზაინი თანმიმდევრული რჩება. ჩემს პროექტში ასეთი ცვლადებია --primary, --surface, --space-md (8px-იანი ბადე: 4/8/16/24/32), --radius-md და სტატუსების ფერები. სწორედ ამ ცვლადებით მუშაობს ღია და მუქი თემა: .dark კლასი მხოლოდ ცვლადების მნიშვნელობებს ცვლის, ხოლო მთავარი ფერის ლურჯიდან ნარინჯისფრად შეცვლას რამდენიმე სტრიქონი დასჭირდა.

8. Flexbox vs CSS Grid

Flexbox ერთგანზომილებიანია და ელემენტებს ერთ ხაზზე (სტრიქონად ან სვეტად) ალაგებს, CSS Grid კი ორგანზომილებიანია და სტრიქონებსა და სვეტებს ერთდროულად მართავს. ჩემს პროექტში Grid გამოვიყენე გვერდის ჩონჩხისთვის (260px 1fr და grid-template-areas: Header, Sidebar, Main, Footer) და ბარათების ბადისთვის (repeat(auto-fit, minmax(190px, 1fr))). Flexbox-ით კი Header ავაწყე (ლოგო მარცხნივ, პროფილი მარჯვნივ) და ბარათების შიგთავსი (იკონა და ტექსტი).

9. UI კომპონენტების ხელახალი გამოყენებადობა

უნივერსალური კომპონენტი ცვალებად ნაწილებს Props-ით იღებს, ამიტომ შიდა კოდის შეცვლა არ სჭირდება. მაგალითად, <Button variant="danger" isLoading> ფერს variant-ით ცვლის, StatusBadge სტატუსს გადაცემული item-იდან თვითონ ითვლის, ხოლო StatCard-ს to prop-ის გადაცემისას ბმულად აქცევს. Modal შიგთავსს children-ით იღებს, ამიტომ ერთი და იგივე მოდალი გამოიყენება ფორმისთვის, ჩამოწერისთვის და წაშლის დადასტურებისთვის — ეს SOLID-ის Open/Closed პრინციპია.

ბლოკი IV: ინტერაქტიულობა, მოდალები, Responsive და Debugging

10. ძიების ოპტიმიზაცია (useDebounce)

Debounce-ის გარეშე ძიება ყოველ აკრეფილ ასოზე ეშვება: სიტყვა „ლეპტოპი“ შვიდ ფილტრაციას გამოიწვევდა, რაც ზედმეტი გამოთვლა და ზედმეტი მოთხოვნებია. useDebounce ყოველ ცვლილებაზე 300ms-იან setTimeout-ს იწყებს, ახალი ასოს აკრეფისას კი cleanup-ში clearTimeout ძველ ტაიმერს აუქმებს. ამიტომ ძიება მხოლოდ მაშინ სრულდება, როცა მომხმარებელი ბეჭდვას 300ms-ით შეწყვეტს, ველში აკრეფილი ტექსტი კი მაშინვე ჩანს.

11. მოდალური ფანჯრები (createPortal)

ReactDOM.createPortal მოდალს იმ ადგილას კი არ ხატავს, სადაც კომპონენტი დგას, არამედ პირდაპირ document.body-ში. ამიტომ მას მშობელი ელემენტის overflow: hidden არ ჭრის და მშობლის z-index-ის კონტექსტი არ ზღუდავს — მოდალი ყოველთვის ყველაფრის, მათ შორის რუკის, თავზე ჩანს, React-ის state და props კი ჩვეულებრივად მუშაობს. ჩემი მოდალი იხურება ×-ით, ფონზე დაჭერით და Escape-ით, ფოკუსიც სწორად იმართება.

12. Fail-Fast ვალიდაცია და Double-Click დაცვა

Fail-Fast ნიშნავს, რომ არასწორი მონაცემი შეცდომას მაშინვე, ფრონტენდზე აჩენს და სერვერამდე არ მიდის. ჩემს პროექტში Zod ამოწმებს SKU-ს ფორმატს, 13-ციფრიან შტრიხკოდს, პაროლის წესებს და იმას, რომ ჩამოსაწერი ან გასაცემი რაოდენობა ნაშთს არ აღემატებოდეს. disabled={isSubmitting} გაგზავნის დროს ღილაკს ბლოკავს, ამიტომ სწრაფი ორმაგი დაჭერა ორ მოთხოვნას ვერ გააგზავნის და არც დუბლიკატი ჩანაწერი შეიქმნება, არც ნაშთი ჩამოიწერება ორჯერ.

13. Responsive Web Design

1440px-ზე მენიუ მუდმივად ჩანს, ცხრილები კი სრულად; 768px-ზე მენიუ იმალება და ცხრილის რიგები ბარათებად იქცევა; 375px-ზე ბადე ერთსვეტიანია, სტატისტიკა 2×2-ზეა, ჰორიზონტალური ზოლები (მაგალითად, TOP 5) კი გვერდულად სქროლდება. მობილურ ნავიგაციას ☰ ჰამბურგერ-მენიუ ემსახურება: მენიუ მარცხნიდან გამოდის (transform: translateX), ფონი მუქდება, ხოლო Escape ან ფონზე დაჭერა მენიუს ხურავს. ტესტირება 3 ბრაუზერსა და 6 მოწყობილობაზე ჩავატარე, ჯამში 90 კომბინაცია.

14. მდგრადობა და Debugging

ErrorBoundary Runtime შეცდომას იჭერს და თეთრი ეკრანის ნაცვლად Fallback-ს აჩვენებს ღილაკით „სცადეთ ხელახლა“, ხოლო Header და მენიუ მუშაობას აგრძელებს (ამის ჩვენება მისამართზე ?crash-ის დამატებით შეიძლება). Offline Fallback: თუ სერვერი შეცდომას აბრუნებს ან ინტერნეტი წყდება, აპლიკაცია localStorage-ში შენახულ (ან დემო) მონაცემებს აჩვენებს და გამაფრთხილებელ ბანერს. DevTools-ში ვიყენებ Breakpoint-ებს (Call Stack, Scope), console.table-ს, console.time-ს და Network ტაბს — ასე ვიპოვე, მაგალითად, Race Condition, როცა ძველი პასუხი ახალს გადაფარავდა.

ბლოკი V: წარმადობა, State Management და ტესტირება

15. Memoization და Code Splitting

useMemo მძიმე გამოთვლის შედეგს იმახსოვრებს (ჩემთან ფილტრაცია, სორტირება და სტატისტიკა) და თავიდან მხოლოდ მონაცემის შეცვლისას ითვლის; useCallback ფუნქციას ყოველ რენდერზე თავიდან არ ქმნის, React.memo კი კომპონენტს ხელახლა არ ხატავს, თუ props არ შეცვლილა — ამიტომ 25-რიგიან ცხრილში ერთი ნივთის რედაქტირებისას მხოლოდ ერთი რიგი გადაიხატება. React.lazy და Suspense თითო გვერდს ცალკე ფაილად (chunk) ტვირთავს მაშინ, როცა მასზე გადავდივართ. ამის შემოღებისას მთავარი JS ფაილი 429 kB-დან 267 kB-მდე შემცირდა (-38%), ახლა კი მძიმე ნაწილები — Leaflet-ის რუკა და დიაგრამები — მხოლოდ საჭირო გვერდზე იტვირთება.

16. მედიაფაილების ოპტიმიზაცია

WebP და AVIF იმავე ხარისხზე JPEG-სა და PNG-ზე გაცილებით მცირე ზომისაა: ჩემს პროექტში 25 ნივთის ფოტო WebP-ში (480×360) ჯამში დაახლოებით 0.5 MB-ია, ანუ ერთი ფოტო საშუალოდ 20 KB. loading="lazy" სურათს მხოლოდ მაშინ ტვირთავს, როცა ის ეკრანს უახლოვდება, ამიტომ გვერდი უფრო სწრაფად იხსნება, width და height კი ჩატვირთვისას „ხტომას“ (CLS) აცილებს. იკონებისთვის SVG-ს (Lucide React) ვიყენებ, რადგან ვექტორულია, ნებისმიერ ზომაზე მკვეთრად ჩანს, მსუბუქია და მისი ფერი CSS-ით იცვლება.

17. გლობალური სთეით-მენეჯმენტი და Persistence

Prop Drilling არის მონაცემის გადაცემა მშობლიდან შვილამდე რამდენიმე დონეზე, მაშინაც კი, როცა შუალედურ კომპონენტებს ეს მონაცემი არ სჭირდებათ; ეს კოდს ართულებს და ცვლილებას რთულს ხდის. პრობლემა გლობალური Store-ით (Context API + useReducer) გადავჭერი: Header, მთავარი გვერდი, მარაგების ცხრილი და მოდალი ერთსა და იმავე მონაცემს useInventory()-ით კითხულობენ, ამიტომ ნივთის დამატებისას Header-ის მრიცხველი მაშინვე 25-დან 26-ზე იცვლება. Store-ის საწყისი მნიშვნელობა localStorage-იდან იკითხება, ყოველი ცვლილება კი useEffect-ით ინახება, ამიტომ გვერდის განახლების შემდეგ როგორც ნივთები, ისე ფილტრები და არჩეული თემა უცვლელი რჩება.

ბლოკი VI: დოკუმენტაცია, ანგარიშგება და პრეზენტაცია

18. Cross-Browser ტესტირება და კოდის ჰიგიენა

ბრაუზერებს სხვადასხვა ძრავი აქვთ (Chrome და Edge Blink-ზეა, Firefox — Gecko-ზე), ამიტომ ერთი და იგივე კოდი შეიძლება სხვადასხვაგვარად მუშაობდეს. ჩემს პროექტში აღმოჩნდა, რომ ქართული სიტყვების სორტირება Chrome-სა და Edge-ში განსხვავდებოდა, და ეს ბრაუზერისგან დამოუკიდებელი ფუნქციით გავასწორე. .gitignore GitHub-ზე არ ტვირთავს node_modules-ს (მძიმეა და npm install-ით აღდგება), dist-ს (build-ის შედეგი) და .env-ს (კონფიდენციალური გასაღებები).

19. README.md vs ტექნიკური ანგარიში

README.md დეველოპერისა და შემფასებლისთვისაა: საიდან ჩანს, რა არის პროექტი და როგორ გაეშვას. სავალდებულო სექციებია: სათაური და Live Demo-ს ბმული, მოკლე აღწერა, ძირითადი ფუნქციონალი, ტექნოლოგიების ცხრილი, საქაღალდეების სტრუქტურა, გაშვების ინსტრუქცია (git clone → npm install → npm run dev), სატესტო ანგარიშები და სკრიპტები. ტექნიკური ანგარიში კი შესრულებულ სამუშაოს შეფასების კრიტერიუმების მიხედვით ასაბუთებს: დაგეგმვა, რისკები, გადაწყვეტილებების დასაბუთება, გადაჭრილი პრობლემები, ტესტირების შედეგები და განვითარების გეგმა (Roadmap).

20. Live Demo და „Demo Effect“

5-წუთიანი დემო ერთ ლოგიკურ Happy Path-ს უნდა მიჰყვებოდეს. ჩემი სცენარია: მენეჯერით შესვლა → მთავარი გვერდი → ახალი შეკვეთა და მისი დადასტურება → საქონლის მიღება საწყობში (მარაგი იზრდება) → გაცემა (მარაგი მცირდება) → მიწოდება რუკაზე → სტატისტიკა → ოპერატორით შესვლა (შეზღუდული მენიუ და „403“) → ErrorBoundary. „Demo Effect“-ის თავიდან ასაცილებლად საიტს წინასწარ ვამოწმებ და ორ ვარიანტს ვამზადებ — Live Demo-ს (Vercel) და ლოკალურს (npm run dev); მზად მაქვს სატესტო ანგარიშები, სცენარი, სქრინშოტები და PDF ანგარიში სარეზერვოდ.